

Service Operation Security & Monitoring

Nhóm 10

Phần 1

Khởi Tạo Môi Trường Production

Môi trường Production



Tính Sẵn Sàng (HA)

Mã nguồn "chạy được" trên máy tính cá nhân là chưa đủ. Môi trường Production đòi hỏi hệ thống phải hoạt động 24/7, có cơ chế tự động phục hồi khi tiến trình gặp lỗi.



Khả Năng Mở Rộng

Cần tách bạch kiến trúc. Application và Database phải được chạy độc lập để dễ dàng cung cấp thêm tài nguyên xử lý khi lượng người dùng tăng đột biến.

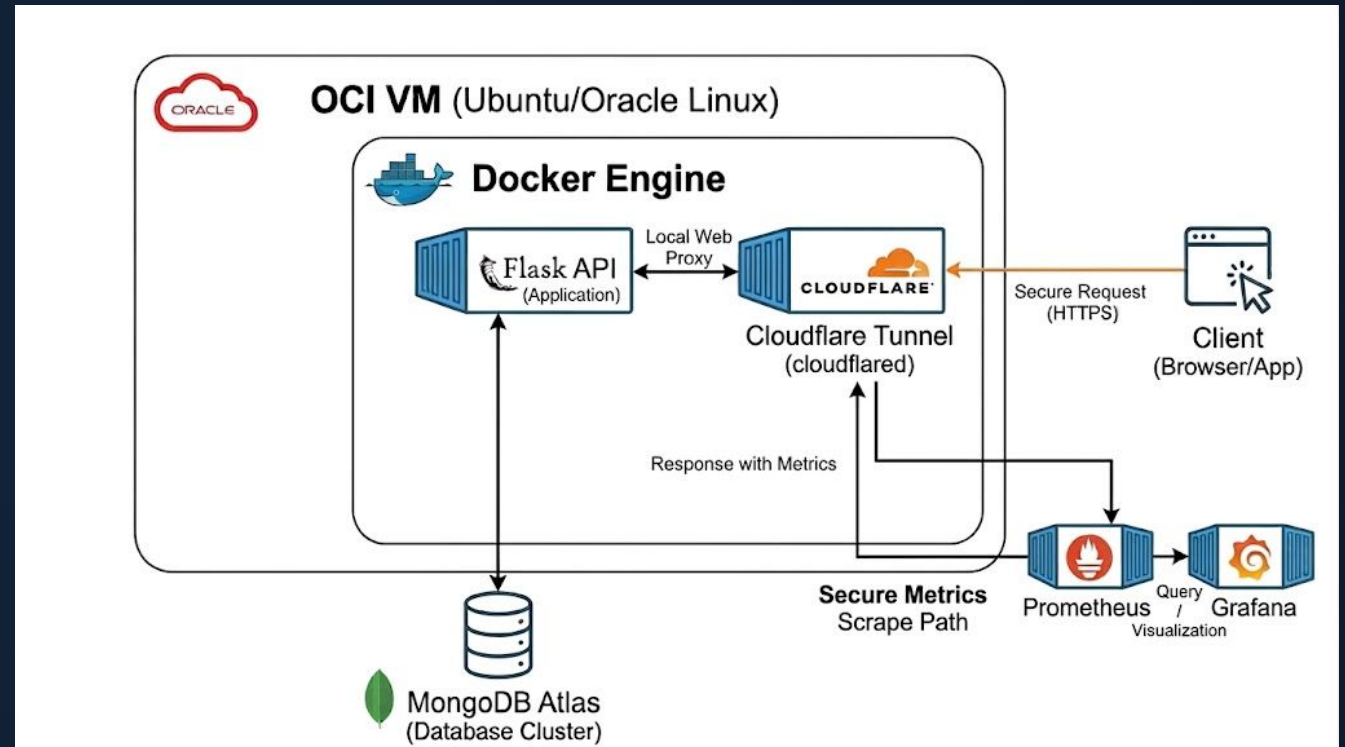


Bảo Mật

Hệ thống đối mặt trực tiếp với môi trường Internet. Cần thiết lập nhiều vòng phòng ngự từ tầng giao thức mạng đến tầng logic ứng dụng.

Kiến trúc tổng thể

-  **API Gateway / Load Balancer:** Phân phối luồng truy cập và chặn các kết nối độc hại.
-  **Application Servers:** Chứa các logic nghiệp vụ cốt lõi.
-  **Database:** Tách biệt hoàn toàn khỏi server ứng dụng để tối ưu hóa truy xuất đĩa (I/O) và an toàn dữ liệu.
-  **Monitoring Stack:** Cấu phần chạy song song, liên tục đo lường của toàn bộ kiến trúc.



Phần 2

Hệ Thống Giám Sát

Ba trụ cột giám sát



Metrics (Đo lường)

Đại diện cho số liệu tổng hợp theo chuỗi thời gian (Time-series). Ví dụ: CPU 80%, 500 req/s. Giúp trả lời ngay lập tức câu hỏi: "**Hệ thống có đang lỗi không?**".



Logs (Ghi chép)

Văn bản ghi lại chi tiết một sự kiện đã xảy ra ở một thời điểm cụ thể. Giúp trả lời câu hỏi: "**Chính xác thì lỗi đó là gì?**".



Tracing (Truy vết)

Truy vết một request khi đi qua hàng loạt các service khác nhau. Giúp trả lời câu hỏi: "**Điểm nghẽn đang nằm ở đoạn code nào?**".

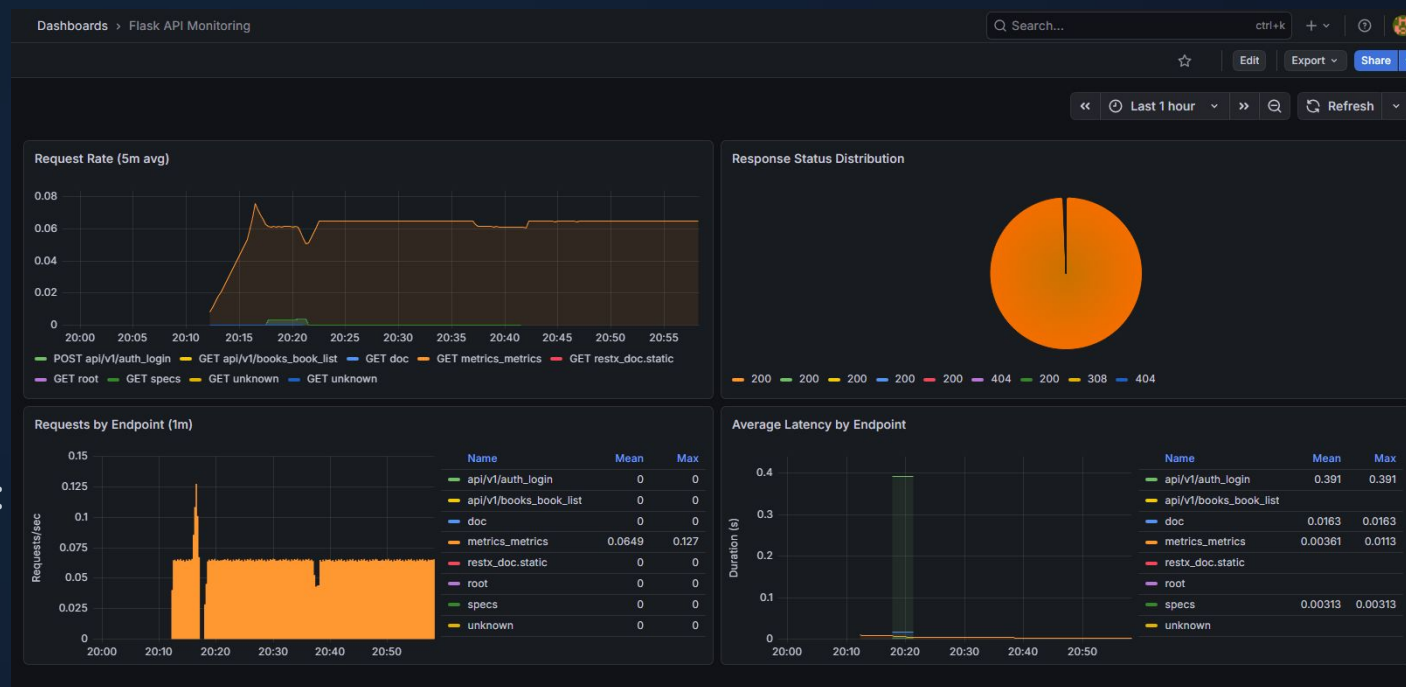
Trực Quan Hoá Metrics

 **Thu thập (Prometheus):** Máy chủ liên tục gọi đến các endpoint `/metrics` của API để kéo dữ liệu thô về lưu trữ định kỳ.

 **Biểu diễn (Grafana):** Chuyển đổi các dữ liệu thô thành những bảng điều khiển sinh động, dễ theo dõi bằng mắt thường.

 **Quy tắc RED:** 3 chỉ số quan trọng của API:

- **Rate:** Tần suất gửi yêu cầu.
- **Errors:** Tỷ lệ phát sinh lỗi hệ thống.
- **Duration:** Độ trễ phản hồi (Latency).



Logging và Tracing



Logging: Log phải được chuẩn hóa ghi ra cả console lẫn file. Mỗi request được đánh thêm Trace ID.



Tracing: Truy vết dựa theo Trace ID, có thể sử dụng các công cụ có sẵn như tail, grep, pipe.

```
2026-05-14 14:41:55,595 - monitoring - INFO - Request completed: GET /metrics - Status: 200 - Trace ID: fd2e5c9b-9f3c-4e23-a232-35f1668e0e1f - Duration: 0.0045s
/app # tail app.log -n 20 | grep "Trace ID"
2026-05-14 14:42:07,999 - monitoring - INFO - Request started: GET /apidocs/ - Trace ID: 15e778c8-4306-4874-b9d6-6651f94fab12
2026-05-14 14:42:08,000 - monitoring - INFO - Request completed: GET /apidocs/ - Status: 200 - Trace ID: 15e778c8-4306-4874-b9d6-6651f94fab12 - Duration: 0.0011s
2026-05-14 14:42:08,179 - monitoring - INFO - Request started: GET /swagger.json - Trace ID: 0fd9b87c-cac7-4917-836c-d1eeaf6ba990
2026-05-14 14:42:08,180 - monitoring - INFO - Request completed: GET /swagger.json - Status: 200 - Trace ID: 0fd9b87c-cac7-4917-836c-d1eeaf6ba990 - Duration: 0.0012s
2026-05-14 14:42:10,790 - monitoring - INFO - Request started: POST /api/v1/auth/login - Trace ID: addcca02-029a-4a2c-b756-5466a2dfcea0
2026-05-14 14:42:11,203 - monitoring - INFO - Request started: GET /metrics - Trace ID: 676581d7-d565-4945-a597-22a5a9127fdc
2026-05-14 14:42:11,208 - monitoring - INFO - Request completed: GET /metrics - Status: 200 - Trace ID: 676581d7-d565-4945-a597-22a5a9127fdc - Duration: 0.0047s
2026-05-14 14:42:11,500 - monitoring - INFO - Request completed: POST /api/v1/auth/login - Status: 200 - Trace ID: addcca02-029a-4a2c-b756-5466a2dfcea0 - Duration: 0.7095s
2026-05-14 14:42:26,205 - monitoring - INFO - Request started: GET /metrics - Trace ID: 959ad3a8-427f-41f8-9c76-2256d3d50615
2026-05-14 14:42:26,210 - monitoring - INFO - Request completed: GET /metrics - Status: 200 - Trace ID: 959ad3a8-427f-41f8-9c76-2256d3d50615 - Duration: 0.0052s
2026-05-14 14:42:42,366 - monitoring - INFO - Request started: GET /metrics - Trace ID: 190c7189-0b80-4122-8a9b-75ed3dd90c24
2026-05-14 14:42:42,371 - monitoring - INFO - Request completed: GET /metrics - Status: 200 - Trace ID: 190c7189-0b80-4122-8a9b-75ed3dd90c24 - Duration: 0.0058s
```

Phần 3

Bảo Mật

Rate Limiting

▼ Rate Limiting

Khái niệm: Đặt ra giới hạn số lượng request tối đa mà một Client được phép gửi trong một khoảng thời gian (VD: 100 req/phút).

Mục đích: Ngăn chặn kẻ xấu dùng Bot để càn quét, dò mật khẩu hoặc cố tình làm sập hệ thống (DDoS). API sẽ tự động trả về mã lỗi HTTP 429 nếu vượt ngưỡng.

```
1 from flask_limiter import Limiter
2 from flask_limiter.util import get_remote_address
3
4 limiter = Limiter(
5     get_remote_address,
6     default_limits=["200 per day", "50 per hour"]
7 )
```

```
@limiter.limit("20 per hour")
@role_required("admin")
@jwt_required()
@ns_books.expect(book_request_model, validate=True)
@ns_books.response(201, 'Created')
@ns_books.response(400, 'Bad Request')
def post(self):
    """Add new book"""
    data = request.json

    new_book = Book(title = data['title'], author = data.get('aut
```

Web Application Firewall



Khái niệm: Tường lửa ứng dụng web chuyên dụng để soi dữ liệu bên trong các gói tin HTTP.

Mục đích: Tự động nhận diện và vứt bỏ các chuỗi ký tự độc hại trước khi chúng có cơ hội chạm vào mã nguồn API. Đặc biệt hiệu quả với SQL Injection hay XSS. Ngoài ra hiện nay còn có thể phát hiện và ngăn chặn AI crawler

